

BÁO CÁO KẾ HOẠCH TRIỂN KHAI VÀ PHÂN TÍCH THIẾT KẾ HỆ THỐNG

Đề tài: CryptoVault - Ứng dụng ví crypto mobile tích hợp NFT Marketplace và P2P Marketplace

1. Mục tiêu tài liệu

Tài liệu này được xây dựng để:

1. Tổng hợp và chuẩn hóa phần **nguyên cứu lý thuyết, phân tích thiết kế, triển khai kỹ thuật**, và **kiểm thử đánh giá** cho hệ thống CryptoVault.
2. Bóc tách hiện trạng dự án từ mã nguồn đã triển khai, làm rõ phần nào đã hoàn thành, phần nào cần hoàn thiện.
3. Cung cấp kế hoạch triển khai theo tuần để phục vụ báo cáo học thuật và nghiệm thu với giảng viên.

Phạm vi phản ánh trong tài liệu bám theo mã nguồn trong workspace:

- Mobile app: /Users/phongva/Code/CryptoVault
 - Backend marketplace: /Users/phongva/Code/CryptoVault/crypto-vault-server
 - Backend P2P: /Users/phongva/Code/CryptoVault/p2p-backend
 - CSDL và script dữ liệu: /Users/phongva/Code/CryptoVault/supabase
 - Tài liệu kiến trúc mở rộng: /Users/phongva/Code/CryptoVault/docs/dex-architecture
-

2. Phần nghiên cứu lý thuyết (nền tảng học thuật)

Phần này chiếm khoảng 30% dung lượng báo cáo, mục tiêu là giải thích nguyên lý công nghệ cốt lõi mà CryptoVault đang sử dụng.

2.1. Mật mã học cơ sở trong ví Web3

CryptoVault hoạt động theo mô hình ví không lưu ký (non-custodial), do đó nền tảng mật mã là điều kiện bắt buộc:

- Người dùng sở hữu cặp khóa công khai/ khóa riêng (Public Key / Private Key).
- Địa chỉ ví được suy ra từ public key.
- Quyền chuyển tài sản gắn với private key, không phụ thuộc tài khoản tập trung của hệ thống.

Với các chain EVM, dự án sử dụng dịch vụ Web3 để thao tác ký và phát giao dịch. Lớp xử lý thể hiện trong:

- `src/core/services/Web3/index.ts`

Với TON, dự án dùng lớp transaction riêng để tạo external message, ước lượng phí và chuẩn bị payload:

- `src/core/services/TonTransactions/tonTransactions.ts`
- `src/core/services/TonTransactions/tonConnectTransfer.ts`

2.2. Chuẩn tạo ví HD: BIP-39, BIP-32, BIP-44

2.2.1. BIP-39 (Mnemonic / Seed Phrase)

- BIP-39 định nghĩa cách sinh cụm từ khôi phục (thường 12 hoặc 24 từ).
- Trong dự án có tệp wordlist BIP-39:
 - `src/assets/wordlistsBIP39.json`
- Luồng tạo ví mới gọi native wallet core để sinh mnemonic:
 - `src/auth/CreateNewWallet/index.view.tsx`

2.2.2. BIP-32/BIP-44 (HD Wallet và Derivation Path)

- Từ một seed phrase có thể sinh ra nhiều khóa con theo cây phân cấp.
- Dự án có logic lấy key/address theo `slip0044` và `derivationPath` cho EVM:
 - `src/core/services/Web3/index.ts`

Ý nghĩa học thuật:

- Chứng minh tính mở rộng tài khoản đa chuỗi từ một seed phrase gốc.
- Hỗ trợ kiến trúc multi-chain mà không yêu cầu người dùng quản lý nhiều cụm khóa độc lập.

2.3. Cơ chế non-custodial và khác biệt với mô hình lưu ký

2.3.1. Mô hình lưu ký (custodial)

- Khóa riêng nằm phía server hoặc bên thứ ba.
- Người dùng phụ thuộc nhà cung cấp dịch vụ.
- Rủi ro tập trung cao, lỗi hoặc tấn công phía server có thể ảnh hưởng diện rộng.

2.3.2. Mô hình không lưu ký (non-custodial) của CryptoVault

- Dữ liệu nhạy cảm được lưu cục bộ:
 - `src/core/services/SecureStorage/index.ts`
- Bật khóa phiên bằng PIN/biometric khi app trở lại foreground:
 - `src/core/hooks/useRequirePinCode.ts`
- App đóng vai trò công cụ điều phối giao dịch; quyền sở hữu tài sản thuộc người dùng.

Kết luận học thuật:

- CryptoVault đi đúng nguyên tắc Web3: **self-custody**, giảm phụ thuộc vào hạ tầng tập trung.

2.4. Blockchain data flow: từ người dùng tới chain

Luồng tổng quát:

1. Người dùng nhập lệnh giao dịch trên giao diện app.
2. App tạo payload giao dịch theo chain đích.
3. Giao dịch được ký cục bộ bởi khóa người dùng.
4. Payload được broadcast tới RPC node.
5. Node xác thực, đưa vào mempool, sau đó ghi vào block.
6. Hệ thống theo dõi trạng thái xác nhận và đồng bộ về dữ liệu hiển thị.

Các thành phần kỹ thuật liên quan:

- RPC fallback để tăng độ tin cậy kết nối:
 - `src/core/utis/rpcUtils.ts`
- Điều khiển mainnet/testnet cho TON:
 - `src/core/utis/tonNetwork.ts`

3. Phần phân tích và thiết kế hệ thống

Mục tiêu phần này là chuyển lý thuyết sang kiến trúc triển khai cụ thể, làm rõ module, luồng dữ liệu, ranh giới trách nhiệm và cơ chế bảo mật.

3.0. Phân tích Actor và chức năng

Phần này mô tả các tác nhân (actor) tương tác với hệ thống và chức năng tương ứng, phục vụ đặc tả use-case và làm cơ sở vẽ UML Use Case Diagram.

3.0.1. Danh sách actor chính

1. Người dùng ví (Wallet User)
2. Người giao dịch NFT (NFT Trader)
3. Dịch vụ blockchain/RPC provider (External System)
4. Dịch vụ dữ liệu bên thứ ba (External API: token metadata, market data)
5. Bộ máy quản lý phí giao dịch (Fee Engine - Internal)

3.0.2. Chức năng của từng actor

Actor 1: Người dùng ví (Wallet User) Chức năng:

1. Tạo ví mới (sinh mnemonic/seed phrase).
2. Thiết lập mã PIN và bật cơ chế khóa ứng dụng.
3. Chọn/chuyển mạng blockchain hỗ trợ.
4. Xem số dư tài sản và lịch sử giao dịch.
5. Thêm token tùy chỉnh bằng contract address.
6. Gửi tài sản, ước lượng phí, ký và phát giao dịch.

Liên kết mã nguồn:

- `src/auth/CreateNewWallet/index.view.tsx`
- `src/core/hooks/useRequirePinCode.ts`
- `src/features/home/addCustomToken/evm/addCustomToken.evm.hook.ts`
- `src/core/services/TonTransactions/tonTransactions.ts`

Actor 2: Người giao dịch NFT (NFT Trader) Chức năng:

1. Kết nối ví và xác thực quyền sở hữu địa chỉ.
2. Tải ảnh NFT lên storage.
3. Tạo metadata NFT.
4. Mint NFT lên mạng TON (testnet/mainnet theo cấu hình).
5. Tạo phiên đấu giá (auction) cho NFT.
6. Tham gia đặt giá (bid) và theo dõi trạng thái phiên.

Liên kết mã nguồn:

- `src/features/marketplace/*`
- `crypto-vault-server/src/routes/marketplaceRoutes.js`
- `ton-contracts/contracts/nft_collection.tact`
- `ton-contracts/contracts/nft_item.tact`
- `ton-contracts/contracts/nft_auction.tact`

Actor 3: Dịch vụ blockchain/RPC provider (External System) Chức năng:

1. Nhận transaction payload đã ký từ ứng dụng.
2. Xử lý broadcast và trả transaction hash.
3. Cung cấp dữ liệu block, account, fee estimation.
4. Trả trạng thái xác nhận giao dịch để app/backend đồng bộ.

Liên kết mã nguồn:

- `src/core/utils/rpcUtils.ts`
- `src/core/services/Web3/index.ts`
- `src/core/services/TonTransactions/tonConnectTransfer.ts`

Actor 4: Dịch vụ dữ liệu bên thứ ba (External API) Chức năng:

1. Cung cấp metadata token theo contract.
2. Cung cấp market data phục vụ hiển thị giá/tài sản.
3. Hỗ trợ dữ liệu bổ sung để làm giàu trải nghiệm UI.

Liên kết mã nguồn:

- `src/features/home/addCustomToken/evm/addCustomToken.evm.hook.ts`
- `src/core/services/Moralis/index.ts`
- `crypto-vault-server/src/services/*`

Actor 5: Bộ máy quản lý phí giao dịch (Fee Engine - Internal) Chức năng:

1. Ước lượng phí mạng (network fee) trước khi gửi giao dịch.
2. Tính phí nền tảng/admin fee theo tỷ lệ cấu hình.
3. Tách cấu phần phí: network fee, admin fee, total fee.
4. Trả kết quả cho UI để người dùng xác nhận trước khi ký và broadcast.
5. Đảm bảo cùng một logic fee cho cả luồng TON transfer và TON Connect.

Liên kết mã nguồn:

- `src/core/services/TonTransactions/tonTransactions.ts`
- `src/core/services/TonTransactions/tonConnectTransfer.ts`
- `src/core/services/TonTransactions/transferUtils.ts`
- `src/features/transfer/ton/ton.transfer.hook.ts`

3.0.3. Ma trận Actor - Chức năng cốt lõi

1. Wallet User: Create Wallet, Secure Access, Transfer, Manage Asset.
2. NFT Trader: Mint NFT, Create Auction, Bid.
3. Blockchain/RPC: Broadcast, Confirm, Provide On-chain Data.
4. External API: Provide Token/Market Metadata.
5. Fee Engine: Estimate Network Fee, Calculate Admin Fee, Return Total Fee.

3.0.4. Gợi ý chuyển thành UML Use Case Khi vẽ biểu đồ Use Case trong báo cáo:

1. Đặt hệ thống trung tâm là **CryptoVault Platform**.
2. Vẽ 5 actor ở ngoài biên hệ thống.
3. Dùng các use-case chính:
 - Create Wallet
 - Switch Network
 - Add Custom Token
 - Send Transaction
 - Estimate Fee
 - Calculate Admin Fee
 - Mint NFT
 - Create Auction
 - Bid Auction
4. Dùng quan hệ include cho các bước chung như:
 - Verify PIN/Biometric
 - Estimate Fee
 - Sign Transaction
 - Broadcast Transaction

3.1. Kiến trúc tổng thể

Hệ thống hiện tại được tách thành 3 lớp chính:

1. **Mobile client (React Native + Expo):**
 - Giao diện người dùng, quản lý session local, ký giao dịch, hiển thị tài sản.
2. **Marketplace backend (crypto-vault-server):**
 - API NFT/auction, market data, tích hợp nguồn dữ liệu bên ngoài.
3. **P2P backend (p2p-backend):**
 - Nghiệp vụ giao dịch P2P tách riêng để dễ kiểm soát rủi ro tài chính.

Kèm theo:

- Supabase/PostgreSQL cho dữ liệu nghiệp vụ.
- Supabase Storage cho tài nguyên media NFT.
- Script schema/seed để dựng môi trường.

3.2. Thiết kế theo module nghiệp vụ

3.2.1. Module A - Định danh và khởi tạo ví Trạng thái hiện tại:

- Đã có luồng tạo ví mới.
- Đã có bước tạo PIN và luồng hiển thị mnemonic.
- Đã có lưu dữ liệu cục bộ qua secure storage.

Tập tham chiếu:

- src/auth/CreateNewWallet/index.view.tsx
- src/auth/ShowPassPhrase/ShowPassPhrase.hook.ts
- src/core/services/SecureStorage/index.ts

Nhận xét thiết kế:

- Luồng onboarding bám đúng nguyên tắc: tạo khóa trước, xác minh người dùng (PIN) trước khi dùng app.
- Tăng độ an toàn nếu bổ sung màn xác thực lại seed phrase bắt buộc trước khi hoàn tất.

3.2.2. Module B - Quản lý đa chuỗi (Multi-chain) Trạng thái hiện tại:

- Có cơ chế chọn protocol/network theo dữ liệu backend.
- Có phân tách testnet/mainnet.
- Có lớp fallback provider cho RPC.

Tập tham chiếu:

- src/features/home/manageProtocol/addProtocol/addProtocol.hook.ts
- src/core/utils/rpcUtils.ts
- src/core/utils/tonNetwork.ts

Nhận xét thiết kế:

- Hướng thiết kế đúng cho scale multi-chain.
- Cần chuẩn hóa ma trận chain support để tránh lệch behavior giữa EVM/TON/Bitcoin.

3.2.3. Module C - Quản lý tài sản (Asset Management) Trạng thái hiện tại:

- Có đọc token info, thêm custom token theo contract.
- Có đồng bộ một phần token metadata lên backend.

Tập tham chiếu:

- src/features/home/addCustomToken/evm/addCustomToken.evm.hook.ts

Nhận xét thiết kế:

- Luồng hợp lý cho MVP: tra cứu metadata -> validate -> thêm vào danh sách tài sản.
- Nên bổ sung quy trình kiểm tra token scam/list deny ở backend.

3.2.4. Module D - Xử lý giao dịch (Transaction) Trạng thái hiện tại:

- TON transfer có luồng estimate fee.
- Có luồng tạo transaction object và chuẩn bị broadcast.
- EVM có service bao gồm ước lượng gas và send transaction.

Tập tham chiếu:

- `src/core/services/TonTransactions/tonTransactions.ts`
- `src/core/services/TonTransactions/tonConnectTransfer.ts`
- `src/core/services/Web3/index.ts`

Nhận xét thiết kế:

- Kiến trúc tách transaction service theo chain là đúng.
- Cần đồng bộ chuẩn error handling để UI hiển thị lỗi nhất quán khi sign/broadcast thất bại.

3.2.5. Module E - Quản lý Fee (Fee Management) Trạng thái hiện tại:

1. Đã có luồng ước lượng phí mạng trước khi gửi (`estimateFee=true`) cho TON.
2. Đã có cấu trúc tính admin fee/tỷ lệ phí nền tảng trong transaction service.
3. Đã có hàm ước tính giá trị gửi tối đa sau khi trừ phí (`estimateMax`).

Tập tham chiếu:

- `src/core/services/TonTransactions/tonTransactions.ts`
- `src/core/services/TonTransactions/tonConnectTransfer.ts`
- `src/core/services/TonTransactions/transferUtils.ts`

Thiết kế quản lý phí đề xuất cho code hiện tại:

1. Chuẩn hóa đối tượng phí trả về UI:
 - `networkFee`
 - `adminFee`
 - `totalFee`
 - `maxSendable`
2. Tách hàm tính phí thành một service dùng chung để tránh lệch giữa transfer và ton connect.
3. Bổ sung rule hiển thị:
 - Nếu fee estimate thất bại: chặn nút gửi.
 - Nếu fee tăng đột biến: bắt buộc người dùng xác nhận lại.
4. Ghi log telemetry ẩn danh cho lỗi estimate/broadcast để cải thiện độ ổn định.

Tiêu chí hoàn thành module quản lý fee:

1. Cùng một input luôn cho ra cùng cấu trúc phí trên mọi màn gửi.
2. Không phát sinh giao dịch khi chưa có kết quả estimate hợp lệ.
3. Người dùng luôn nhìn thấy tổng chi phí trước khi ký.

3.3. Thiết kế bảo mật

Các lớp bảo mật đã có:

1. **Local secure storage** cho dữ liệu nhạy cảm.
2. **PIN/biometric gating** khi quay lại app.
3. **Network segmentation** giữa backend marketplace và backend P2P.
4. **Mode testnet ưu tiên** cho môi trường thử nghiệm.

Điểm cần cải thiện:

1. Rà soát việc log thông tin nhạy cảm ở môi trường debug.
2. Bổ sung threat model cho các luồng transfer và quản lý tài sản.

3. Áp dụng chính sách rotate secret và hardening env cho deploy production.

3.4. Biểu đồ tuần tự khuyến nghị (mô tả text)

3.4.1. Sequence: Ký và phát giao dịch TON

1. User nhập địa chỉ nhận + số lượng.
2. App gọi `createTransfer(..., estimateFee=true)` để mô phỏng phí.
3. App hiển thị phí và yêu cầu xác nhận.
4. User nhập PIN hoặc xác thực biometric.
5. App tạo payload đã ký.
6. App/connector broadcast payload lên mạng TON.
7. App nhận tx hash và cập nhật lịch sử giao dịch.

3.4.2. Sequence: Add custom token EVM

1. User nhập contract address.
 2. App gọi API token metadata (Moralis/service).
 3. App validate trùng token trong danh sách hiện có.
 4. App dispatch thêm token vào state local.
 5. App gửi request backend để quản lý tập trung metadata token custom.
-

4. Phần triển khai kỹ thuật (xây dựng sản phẩm)

4.1. Công nghệ và stack triển khai

4.1.1. Mobile

- React Native 0.74.x
- Expo SDK 51
- Redux Toolkit
- ethers, web3, TON SDK liên quan

Tham chiếu:

- `package.json` (root project)

4.1.2. Backend marketplace

- Node.js + Express
- PostgreSQL
- Redis/BullMQ (phục vụ worker hàng đợi ở một số luồng)

Tham chiếu:

- `crypto-vault-server/package.json`

4.1.3. Backend P2P

- TypeScript + Express
- Zod validate schema
- Supabase SDK

Tham chiếu:

- `p2p-backend/package.json`

4.2. Tổ chức source code

4.2.1. Mobile app

- `src/auth`: onboarding, tạo ví, pin code.
- `src/core/services`: service blockchain, bảo mật, network, analytics.
- `src/features`: các tính năng màn hình theo domain (home, transfer, marketplace, swap...).

4.2.2. Backend marketplace

- API routes cho NFT, auction, admin, public feed.
- Service lớp nghiệp vụ tách riêng theo tích hợp.

4.2.3. Backend P2P

- Mô hình controller-service-repository rõ ràng.
- Có middleware auth và guard secret cho job route.

4.3. Môi trường triển khai

4.3.1. Local/dev

- Chạy mobile bằng Expo.
- Chạy backend marketplace và P2P độc lập.
- Dùng Supabase schema local/remote theo env.

4.3.2. Docker

- Có `docker-compose.yml` cho backend + admin dashboard.
- Có `docker-compose.prod.yml` cho image production.
- Có script build/push image:
 - `deploy_docker.sh`

4.3.3. Cloud deploy

- Có cấu hình Render:
 - `render.yaml`

4.4. Kế hoạch triển khai chi tiết theo tuần (tuần tới)

Mốc thời gian tham chiếu: **01/06/2026 - 07/06/2026**.

Tuần 1 (02/06/2026): Hoàn thiện module ví Mục tiêu:

- Kiểm tra toàn bộ flow PIN/biometric lock.
- Khóa scope báo cáo và demo.
- Chốt danh sách chức năng bắt buộc.

Việc thực hiện:

1. Freeze branch báo cáo.
2. Chốt matrix tính năng:
 - Create wallet
 - Switch network
 - Add custom token
 - Send transaction
 - NFT/P2P luồng demo
3. Chuẩn hóa env mẫu từ `.env.example`.

Deliverable:

- Bảng scope khóa.
 - Danh sách issue theo mức độ ưu tiên.
1. Tối ưu flow onboarding và tạo ví mới.
 2. Rà soát bảo mật mnemonic và chính sách hiển thị/copy.
 3. Test onboarding: create -> show phrase -> lock/unlock.

Deliverable:

- Biên bản test module ví.

Ngày 3 (08/06/2026): Củng cố multi-chain và tài sản Mục tiêu:

- Ổn định chọn network/protocol.
- Hoàn thiện thêm token custom + kiểm tra duplicate.

Việc thực hiện:

1. Test fallback RPC theo chain.
2. Kiểm thử add custom token với contract thật trên testnet.
3. Chuẩn hóa lỗi UI khi metadata token không hợp lệ.

Deliverable:

- Danh sách test case module asset.
- Video/screenshot minh chứng.

Ngày 4 (15/06/2026): Luồng giao dịch Mục tiêu:

- Kiểm tra đầy đủ luồng estimate -> sign -> broadcast.

Việc thực hiện:

1. TON transfer testnet với nhiều mức fee.
2. EVM send transaction qua network testnet (Sepolia/BSC Testnet tùy cấu hình).
3. Log và chuẩn hóa thông báo lỗi thất bại.

Deliverable:

- Tx hash testnet.
- Checklist pass/fail cho transaction flow.

Ngày 5 (22/06/2026): Security và hardening MVP Mục tiêu:

- Rà soát rủi ro bảo mật mức ứng dụng.

Việc thực hiện:

1. Kiểm tra dữ liệu nhạy cảm trong log.
2. Kiểm tra khóa app khi foreground.
3. Review các endpoint nhạy cảm ở P2P/backend.

Deliverable:

- Security checklist.
- Danh sách đề xuất hardening.

Ngày 6 (29/06/2026): Regression và tài liệu hóa Mục tiêu:

- Chạy regression test.
- Chốt báo cáo và minh chứng.

Việc thực hiện:

1. Re-test toàn bộ module chính.
2. Cập nhật biểu đồ kiến trúc và sequence cuối.
3. Chuẩn hóa ngôn ngữ báo cáo gửi giảng viên.

Deliverable:

- Bản báo cáo gần-final.
- Bộ ảnh minh chứng và tx sample.

Ngày 7 (04/07/2026): Tổng duyệt và nộp Mục tiêu:

- Hoàn tất hồ sơ nộp.

Việc thực hiện:

1. Chốt phiên bản PDF/DOCX.
2. Soát chính tả, format, mục lục.
3. Nộp báo cáo + chuẩn bị script demo.

Deliverable:

- Bản nộp chính thức.
 - Kịch bản demo 5-10 phút.
-

5. Phần kiểm thử và đánh giá

5.1. Chiến lược kiểm thử

Áp dụng 4 lớp:

1. **Functional test**: theo module nghiệp vụ.
2. **Integration test**: mobile ↔ backend ↔ blockchain API.
3. **Security-oriented test**: lock screen, storage, validation đầu vào.
4. **UAT demo test**: theo kịch bản thực tế trước khi nộp.

5.2. Bộ test case tối thiểu đề xuất

Nhóm ví và bảo mật

1. Tạo ví mới thành công, hiển thị đúng 12 từ.
2. Nhập PIN sai nhiều lần, app xử lý đúng chính sách.
3. App vào background và quay lại: bắt buộc xác thực lại.

Nhóm đa chuỗi và tài sản

1. Chuyển network và làm mới dữ liệu token.
2. Add custom token hợp lệ.
3. Add lại token đã có -> báo duplicate.

Nhóm giao dịch

1. Estimate phí thành công trước khi gửi.
2. Ký giao dịch thành công và trả tx hash.
3. Broadcast lỗi -> hiển thị lỗi chuẩn hóa.

5.3. Tiêu chí đánh giá hoàn thành

Hoàn thành kỹ thuật khi:

1. 100% luồng demo bắt buộc chạy được trên testnet.
 2. Không còn blocker mức P1 ở module ví/giao dịch.
 3. Có minh chứng giao dịch thật bằng tx hash.
 4. Báo cáo mô tả đúng hiện trạng (không ghi vượt mức thực tế code).
-

6. Đánh giá hiện trạng dự án (as-is)

6.1. Điểm mạnh

1. Kiến trúc có phân lớp tương đối rõ và có định hướng production.
2. Đã có nhiều module thực chiến: wallet, transfer, token, NFT/P2P.
3. Có tài liệu kiến trúc mở rộng trong docs/dex-architecture.
4. Có nền tảng deploy Docker/Render và script hỗ trợ.

6.2. Điểm cần hoàn thiện

1. Chất lượng test tự động chưa đồng đều giữa các service.
2. Cần chuẩn hóa tài liệu UML/sequence theo đúng rubric môn học.
3. Cần bổ sung thêm hardening bảo mật trước khi gọi là production-ready.

6.3. Rủi ro kỹ thuật chính

1. Rủi ro trải nghiệm khi RPC timeout nếu fallback chưa phủ đủ chain.
 2. Rủi ro sai khác dữ liệu giữa on-chain và local cache nếu sync chưa chặt.
-

7. Kế hoạch hoàn thiện sau nộp báo cáo

7.1. Giai đoạn 1 (2-4 tuần)

1. Chuẩn hóa error code toàn hệ thống.
2. Bổ sung telemetry cho luồng ký và phát giao dịch.

7.2. Giai đoạn 2 (1-2 tháng)

1. Bổ sung CI/CD kiểm thử tự động.
2. Tăng cường security checklist theo OWASP Mobile.
3. Cải tiến dashboard giám sát backend realtime.

7.3. Giai đoạn 3 (sau MVP)

1. Mở rộng chain support có kiểm soát.
 2. Tối ưu transaction state machine cho UX.
 3. Chuẩn bị kiến trúc cho production scale lớn.
-

8. Kết luận

CryptoVault đã xây dựng được nền tảng kỹ thuật đáng kể cho một ứng dụng ví crypto mobile đa chức năng, gồm quản lý ví, tài sản, giao dịch, NFT marketplace và P2P backend tách lớp. Hướng thiết kế hiện tại phù hợp với định hướng non-custodial và mở rộng đa chuỗi.

Để đạt mức hoàn thiện tốt cho báo cáo học thuật và trình diễn với giảng viên trong tuần tới, trọng tâm cần tập trung vào:

1. Chốt và kiểm thử đầy đủ các luồng transaction testnet.
2. Chuẩn hóa tài liệu thiết kế (kiến trúc + sequence) đúng theo rubric.
3. Cung cấp minh chứng kỹ thuật rõ ràng (tx hash, checklist, ảnh luồng).

Với kế hoạch triển khai 7 ngày nêu trên, dự án có thể đạt trạng thái “MVP hoàn chỉnh để nghiệm thu học phần”, đồng thời giữ nền tảng tốt cho các bước nâng cấp production tiếp theo.